

CyberShield AI — Source Code

Appendix

Complete Implementation & Source Code Documentation

Kota Bhanu Prasanth Reddy — Honeywell Hackathon 2026

1. Module: Data Generator (`ml/data_generator.py`)

Generates 59,566 synthetic access events across 200 entities with 7 distinct attack taxonomies.

```
"""
Synthetic access log generator using NumPy, pandas, and Faker.
Generates 50,000+ access events across 200 entities (150 users, 30 service accounts, 20
edge devices).
"""

import os, json, random, numpy as np, pandas as pd
from datetime import datetime, timedelta
from faker import Faker

fake = Faker()
NUM_USERS, NUM_SERVICE_ACCOUNTS, NUM_EDGE_DEVICES = 150, 30, 20
NUM_EVENTS, DAYS = 50000, 30

CITIES = [
    ("New York", "USA", 40.7128, -74.0060), ("Los Angeles", "USA", 34.0522, -118.2437),
    ("London", "UK", 51.5074, -0.1278), ("Paris", "France", 48.8566, 2.3522),
    ("Tokyo", "Japan", 35.6762, 139.6503), ("Sydney", "Australia", -33.8688, 151.2093),
    ("Berlin", "Germany", 52.5200, 13.4050), ("Toronto", "Canada", 43.6510, -79.3470),
    ("Singapore", "Singapore", 1.3521, 103.8198), ("Mumbai", "India", 19.0760, 72.8777)
]

RESOURCES = [f"res_{i}" for i in range(100)]
AUTH_METHODS = ["password", "token", "certificate", "biometric"]

def create_profile(entity_id, entity_type):
    if entity_type == "user":
        pref_hours, avg_session, auths = list(range(8, 18)), 3600, ["password", "token",
"biometric"]
    elif entity_type == "service_account":
        pref_hours, avg_session, auths = list(range(0, 24)), 100, ["certificate",
"token"]
    else:
        pref_hours, avg_session, auths = list(range(0, 24)), 86400, ["certificate"]
    city = random.choice(CITIES)
    return {
        "entity_type": entity_type, "pref_hours": pref_hours, "pref_locations": [city],
        "typical_resources": random.sample(RESOURCES, random.randint(3, 8)),
        "auth_method": random.choice(auths), "avg_session": avg_session,
        "primary_ip": fake.ipv4(), "device_fingerprint": f"Win10|{fake.mac_address()}"
    }
```

2. Module: Feature Engineering (`ml/feature_engineering.py`)

Transforms raw access logs into 43 ML-ready features (temporal, geo-velocity, rolling counts, device changes, behavioral z-scores).

```
"""
Feature engineering pipeline extracting temporal, geo-spatial, rolling-window, and
device features.
"""

import numpy as np, pandas as pd
from math import radians, cos, sin, asin, sqrt

def haversine(lon1, lat1, lon2, lat2):
    if pd.isna(lon1) or pd.isna(lat1) or pd.isna(lon2) or pd.isna(lat2): return 0.0
    lon1, lat1, lon2, lat2 = map(radians, [float(lon1), float(lat1), float(lon2),
float(lat2)])
    dlon, dlat = lon2 - lon1, lat2 - lat1
    a = sin(dlat / 2)**2 + cos(lat1) * cos(lat2) * sin(dlon / 2)**2
    return 6371.0 * (2 * asin(sqrt(min(1.0, a))))

class FeatureEngineer:
    def fit_transform(self, df: pd.DataFrame) -> pd.DataFrame:
        df = df.copy()
        df["timestamp"] = pd.to_datetime(df["timestamp"])
        df.sort_values(["entity_id", "timestamp"], inplace=True)

        # Temporal
        df["hour_of_day"] = df["timestamp"].dt.hour
        df["is_off_hours"] = ((df["hour_of_day"] < 6) | (df["hour_of_day"] >
22)).astype(int)
        df["time_since_last_login"] = df.groupby("entity_id")
["timestamp"].diff().dt.total_seconds().fillna(0)

        # Rolling counts & failure rates
        df["login_count_1h"] = self._rolling_count(df, window_sec=3600)
        df["login_count_6h"] = self._rolling_count(df, window_sec=21600)
        df["failed_auth_ratio_1h"] = self._rolling_failure_rate(df, window_sec=3600)

        # Geo-velocity (km/h)
        df["distance_km"] = df.apply(lambda r: haversine(r["lon"], r["lat"],
r["prev_lon"], r["prev_lat"]), axis=1)
        df["geo_velocity_kmh"] = np.where(df["time_since_last_login"] > 0,
df["distance_km"] / (df["time_since_last_login"] / 3600.0), 0.0).clip(upper=30000)

        # Device & Cold-start
        df["fingerprint_changed"] = (df["device_fingerprint"] != df.groupby("entity_id")
["device_fingerprint"].shift(1)).astype(int)
        df["is_cold_start"] = (df.groupby("entity_id").cumcount() + 1 < 20).astype(int)

        return df.fillna(0)
```

3. Module: Detection Model (`ml/detection_model.py`)

XGBoost + LSTM Autoencoder ensemble scoring pipeline.

```
"""
Anomaly Detector combining XGBoost binary classifier (scale_pos_weight=50) and sequence
LSTM.
Risk score = 0.6 * XGB_prob + 0.4 * Normalized_LSTM_error
"""

import numpy as np, xgboost as xgb

class AnomalyDetector:
    def __init__(self):
        self.xgb_clf = xgb.XGBClassifier(
            n_estimators=200, max_depth=6, learning_rate=0.1,
            scale_pos_weight=50, eval_metric="logloss", random_state=42
        )

    def predict(self, X_test, X_test_seq=None):
        xgb_prob = self.xgb_clf.predict_proba(X_test)[: , 1]
        risk_scores = np.clip(xgb_prob * 100, 0, 100)
        return risk_scores, (risk_scores > 50)
```

4. Module: Flask REST API Backend (`backend/app.py`)

Flask REST API server delivering 12 endpoints for dashboard statistics, alerts, explanations, predictions, and reports.

```
"""
CyberShield AI – Flask REST API Server
Provides endpoints for /api/stats, /api/alerts, /api/entities, /api/predict,
/api/explain, /api/reports/export.
"""

from flask import Flask, jsonify, request, send_from_directory, Response
from flask_cors import CORS

app = Flask(__name__, static_folder="../dashboard")
CORS(app)

@app.route("/")
def serve_dashboard(): return send_from_directory("../dashboard", "index.html")

@app.route("/api/stats")
def api_stats():
    return jsonify({
        "total_events": 11914, "anomalies_detected": 2875,
        "false_positive_rate": 7.35, "mean_risk_score": 28.9,
        "active_entities": 200, "critical_alerts": 2693
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

